

Introduction

Bitwise XOR problems have a reputation for looking harder than they are, and LeetCode 3702, Longest Subsequence With Non-Zero XOR, is a perfect example. On first read it sounds like a subsequence optimization problem, the kind that usually pulls out dynamic programming or backtracking. It isn't. The entire problem collapses into one observation about how XOR behaves across an array, and once you see it, the solution is a single pass with no extra memory.

Interviewers like this problem because it tests something more specific than "do you know XOR." It tests whether you notice that a brute-force subsequence search is exponential before you start writing one, and whether you can reduce a combinatorial question to a property of the whole array. That instinct — stepping back from the subsequence and asking what the total XOR tells you — is the actual skill being evaluated.

Learning this pattern pays off beyond this one problem. Anywhere duplicates need to be detected instantly, in hashing schemes, checksums, or streaming deduplication, this same cancellation property of XOR shows up.

Problem Overview

You're given an array of integers. You need to choose a subsequence, meaning you pick elements while keeping their original order, but you can skip any of them. The one requirement: XOR every chosen element together, and the result must not be zero. Among all subsequences that satisfy this, return the length of the longest one. If no subsequence can avoid an XOR of zero, return 0.

The key word is subsequence, not subarray. You are allowed to skip elements freely; you just can't reorder them. That freedom is exactly what makes the "drop one element" trick possible later.

Example

Take `nums = [1, 2, 3]`.

- Picking `{2, 3}` gives `2 XOR 3 = 1`, which is nonzero. Valid, length 2.
- Picking all three, `1 XOR 2 XOR 3`, also gives `0`. Wait — let's check: `1 XOR 2 = 3`, then `3 XOR 3 = 0`. That's zero, so the full array fails.

So for `[1, 2, 3]`, the best you can do is drop one element. Removing `1` leaves `{2, 3}`, XOR is `1`, nonzero, length 2. The answer is 2.

Now take `nums = [2, 3, 4]`. XOR them all: `2 XOR 3 = 1`, `1 XOR 4 = 5`. The total is 5, which is nonzero, so the entire array already qualifies. The answer is 3, no removal needed.

Compare that to `nums = [0, 0]`. Every subsequence you can form, `{0}`, `{0}`, or `{0, 0}`, XORs to zero. There is no way out. The answer is 0.

Intuition

The instinct most people reach for first is pairing: try to figure out which specific elements cancel each other out, maybe with a hash set tracking values seen so far, maybe with some DP state tracking achievable XOR values. That instinct is what leads straight into an exponential or at best polynomial-in-value-range solution, and it's solving a harder problem than the one that's actually being asked.

The reframe: instead of asking "which subsequence has nonzero XOR," ask "what does the XOR of the entire array tell me?"

There are only two possibilities for the total XOR of the full array: it's nonzero, or it's zero.

If it's nonzero, you're done immediately. The full array itself is a valid answer, and you can't do better than including everything.

If it's zero, that means the elements cancel out in aggregate. But here's the useful fact: `x XOR x = 0`. If you remove any single nonzero value from a set of elements that XORs to zero, the running total changes from `0` to that value, which is nonzero by definition. So dropping exactly one nonzero element flips the whole array's XOR from zero to nonzero, at the cost of a single position. That gives you `n - 1`.

The only way this fails completely is if there's no nonzero element to drop; that is, every element in the array is `0`. In that case, every subsequence, no matter which elements you pick, XORs to `0`, because `0 XOR 0 XOR ... XOR 0` is always `0`. That's the one edge case that breaks a naive "just subtract one" answer.

So the whole problem reduces to three checks: is the total nonzero, is there at least one nonzero element to remove, or is everything zero.

Brute Force Solution

Idea: Enumerate every possible subsequence, compute its XOR, and track the longest one whose XOR isn't zero.

Algorithm: For an array of length n , there are 2^n subsequences (including the empty one). Generate each, either via recursion (include/exclude each element) or by iterating over bitmasks from 0 to $2^n - 1$, compute the XOR of the selected elements, and keep the maximum length among those with nonzero XOR.

Advantages: Trivial to reason about and verify by hand on small inputs. Good as a warm-up to build intuition about what "cancellation" means before reaching for the optimal trick.

Disadvantages: Completely impractical past a few dozen elements. LeetCode 3702 allows up to 100,000 elements, and 2^{100000} isn't a large number in the normal sense, it's larger than the estimated number of atoms in the observable universe.

Complexity: Time $O(2^n \cdot n)$ (generating each subsequence and XORing its elements), Space $O(n)$ for recursion depth or $O(1)$ extra if iterating bitmasks without storing subsequences.

Optimal Solution

The optimal approach uses exactly the reduction from the intuition section, formalized into three steps:

1. Compute `total = nums[0] XOR nums[1] XOR ... XOR nums[n-1]` in one pass.
2. If `total != 0`, the whole array is valid. Return `n`.
3. If `total == 0`, check whether there's any nonzero element in the array.
4. If yes, return `n - 1` (drop any one nonzero element).
5. If no (every element is `0`), return `0`.

Case Total XOR Nonzero element exists? Answer

A	nonzero	—	<code>n</code>
B	zero	yes	<code>n - 1</code>
C	zero	no (all zeros)	<code>0</code>

It doesn't matter *which* nonzero element you drop in case B; any single nonzero value, once removed, flips the running XOR away from zero. There's nothing to optimize there, so don't spend time picking a "best" one to remove.

Python Code

```

from functools import reduce
from operator import xor

def longest_subsequence(nums: list[int]) -> int:
    total = reduce(xor, nums, 0)

    if total != 0:
        return len(nums)

    if any(nums): # at least one nonzero element
        return len(nums) - 1

    return 0

```

Code Walkthrough

- `reduce(xor, nums, 0)` XORs every element in the array together in one pass, starting from `0`. This replaces a manual loop with `total = 0; for x in nums: total ^= x`, but does the same thing.
- `if total != 0: return len(nums)` handles case A: the full array already satisfies the constraint.
- `any(nums)` returns `True` if at least one element is truthy, meaning nonzero, since Python treats `0` as falsy and any other int as truthy. This distinguishes case B (drop one, return `n - 1`) from case C (all zeros, return `0`).
- The function never builds any subsequence explicitly; it only reasons about the array's aggregate XOR and the presence of a nonzero value.

Dry Run

```
nums = [1, 2, 3]
```

1. `total = 0 XOR 1 = 1`
2. `total = 1 XOR 2 = 3`
3. `total = 3 XOR 3 = 0`
4. `total == 0`, so check `any(nums)`. `1` and `2` are nonzero, so `any(nums)` is `True`.
5. Return `len(nums) - 1 = 2`.

This matches the example: `{2, 3}` is a valid length-2 subsequence, and the full array fails because its total XOR is zero.

Complexity Analysis

Time: $O(n)$. The `reduce` call is one pass, and `any(nums)` in the worst case is another pass, but both are linear and don't nest, so the total stays $O(n)$.

Space: $O(1)$. Only a running XOR total and no auxiliary array, hash set, or DP table is needed. This is optimal, since you have to read every element at least once to compute the total XOR, so $O(n)$ time is a hard lower bound, and no algorithm can do better than $O(1)$ extra space for this problem.

Alternative Solutions

A DP-based approach, tracking, for each prefix, the maximum length of a subsequence achieving each possible XOR value, would also work but is unnecessary here. It costs $O(n \cdot V)$ time and space, where V is the range of possible XOR values, dramatically worse than $O(n)$ time and $O(1)$ space for no benefit. It's only worth mentioning as the "obvious but wrong" instinct: the moment you see "subsequence" and "XOR" together, DP is a natural first thought, but this problem's structure, only one XOR value (zero) ever needs to be avoided, means DP is solving a harder, more general question than what's actually asked.

Edge Cases

- **Empty array:** Not a valid input per constraints, but if encountered, `total` stays `0` and `any([])` is `False`, correctly returning `0`.
- **Single element, nonzero:** `total` equals that element, which is nonzero, so return `1`.
- **Single element, zero:** `total` is `0`, `any([0])` is `False`, return `0`.
- **All zeros:** Handled explicitly by case C; every subsequence XORs to zero.
- **Duplicates that aren't all zero:** e.g. `[5, 5, 3]`. Total is `5 XOR 5 XOR 3 = 3`, nonzero, so the full array of length 3 is valid despite the duplicate pair.
- **Large values (near constraint max):** XOR is bitwise and independent of magnitude, so this doesn't affect correctness or performance.

Common Mistakes

1. **Forgetting the all-zero edge case** and returning `n - 1` even when every element is zero, when it must return `0`.
2. **Trying to figure out which specific element to drop** as if it mattered, when any nonzero element works identically.

3. **Reaching for DP or a hash set of prefix XOR values** by pattern-matching on "subsequence problem" instead of checking the whole-array case first.
4. **Confusing subsequence with subarray** and assuming elements must be contiguous, which would require a completely different (and unnecessary) sliding-window approach.
5. **Not short-circuiting** when `total != 0`, and running unnecessary extra checks (like scanning for a nonzero element) even though the full array already answers the question.

Interview Questions

- What if the problem asked for the longest **subarray** (contiguous) instead of subsequence? How would the approach change?
- Can you extend this to find the longest subsequence whose XOR equals some target `k`, not just "nonzero"?
- What's the maximum possible XOR of any subsequence, and how would you compute it (hint: linear basis / XOR basis)?
- How would you handle this problem in a streaming context where you can't store the whole array?
- Why is `any(nums)` sufficient to check for a nonzero element instead of scanning with a loop, and is there a performance difference?

Similar Problems

- **LeetCode 1442 – Count Triplets That Can Form Two Arrays of Equal XOR:** Also relies on the property that a range XORing to zero means two prefix XORs are equal.
- **LeetCode 1720 – Decode XORed Array:** Builds on prefix-XOR reconstruction, a related but distinct XOR property.
- **LeetCode 1310 – XOR Queries of a Subarray:** Uses prefix XOR arrays, good practice for the same "aggregate operation" mindset.
- **LeetCode 1734 – Decode XORed Permutation:** Another problem where the whole-array XOR unlocks the individual values.

These are grouped because they all hinge on the same core fact: XOR of a range can often be reasoned about via a single running total rather than examining every element combination.

Key Takeaways

- When a subsequence problem's validity depends on one combined operation (XOR, sum, product), check the whole-array case first before reaching for DP or brute force.
- $x \text{ XOR } x = 0$ means duplicates cancel; a zero-XOR subsequence is cancellation in disguise, not some deeper structural property.
- If the total XOR is already nonzero, the full array is your answer. If it's zero, dropping any one nonzero element fixes it in $O(1)$ extra work.
- The only true dead end is an all-zero array, where no subsequence can ever produce a nonzero XOR.
- This reduces an apparently exponential subsequence search to a single $O(n)$ pass with $O(1)$ space.

Watch the Video

If you want to see this reasoning built up step by step, including the dry run and the quiz at the end, watch the full walkthrough here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series that takes scary-looking whiteboard problems and shows the one-line trick usually hiding underneath. New daily drops cover LeetCode problems, core data structures, and the intuition interviewers actually care about, not just the code that passes the test cases.

Call To Action

If this saved you from writing an unnecessary DP table, like the video on YouTube and subscribe so the next daily drop lands in your feed. Subscribe to the Substack for the written breakdowns like this one, drop a comment with your quiz answers or questions, and share this with anyone prepping for coding interviews.

Quick Review

You see array elements with total XOR = 0. What's the pattern to spot?

Removing any one element makes subset XOR nonzero — answer is $n-1$, else it's n .

Time complexity for solving this problem?

$O(n)$ — one pass to XOR everything, no need to check every subsequence.

Space complexity needed?

$O(1)$ — just a running XOR accumulator, no extra structures.

What's the trickiest edge case here?

All elements XOR to 0 but array has only 1 element (which is 0) — no valid subsequence exists, tricky base case.

Why is checking every subsequence the wrong instinct?

Exponential 2^n brute force; the whole-array XOR check collapses it to one linear scan.

Interview follow-up: why does dropping one element always work when total XOR is 0?

Since full XOR is 0, dropping any single nonzero element leaves the rest XOR to that element's value, which is nonzero.